

WT LAB-7

ID:N210264

NAME:A.SREE HARSHA

ROLL :22

CODE:

Index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Student Records Management</title>
```

```
  <link
```

```
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
```

```
  <link rel="stylesheet" href="styles.css">
```

```
</head>
```

```
<body>
```

```
  <div class="container mt-4">
```

```
    <h1 class="text-center mb-4">Student Records Management System</h1>
```

```
    <!-- Database Creation Section -->
```

```
    <div class="card mb-4">
```

```
      <div class="card-header">
```

```
<h3>Phase 1: Database & Data Creation</h3>

</div>

<div class="card-body">

    <button onclick="initializeDatabase()" class="btn btn-primary mb-2">Initialize Database with Sample Data</button>

</div>

</div>

<!-- Query Section -->

<div class="card mb-4">

    <div class="card-header">

        <h3>Phase 2: Queries</h3>

    </div>

    <div class="card-body">

        <div class="row">

            <div class="col-md-6">

                <h4>Retrieval Tasks</h4>

                <button onclick="getAllStudents()" class="btn btn-info mb-2 w-100">4. Get All Students</button>

                <button onclick="getCSStudents()" class="btn btn-info mb-2 w-100">5. Get CS Students</button>

                <button onclick="getHighMathScorers()" class="btn btn-info mb-2 w-100">6. Math Score > 80</button>

                <button onclick="getOlderStudents()" class="btn btn-info mb-2 w-100">7. Age > 21</button>

                <button onclick="getRecentEnrollments()" class="btn btn-info mb-2 w-100">8. Recent Enrollments</button>

            </div>

        </div>

    </div>

</div>
```

```
<div class="col-md-6">
```

```
    <h4>Update Tasks</h4>
```

```
    <button onclick="increaseMathMarks()" class="btn btn-warning  
mb-2 w-100">9. Increase Math Marks</button>
```

```
    <button onclick="addActiveStatus()" class="btn btn-warning  
mb-2 w-100">10. Add Active Status</button>
```

```
    <button onclick="changeDepartment()" class="btn btn-warning  
mb-2 w-100">11. Change Department</button>
```

```
</div>
```

```
</div>
```

```
<div class="row mt-3">
```

```
    <div class="col-md-6">
```

```
        <h4>Delete Tasks</h4>
```

```
        <button onclick="deleteLowestScience()" class="btn btn-danger  
mb-2 w-100">12. Delete Lowest Science</button>
```

```
        <button onclick="deleteOldEnrollments()" class="btn btn-  
danger mb-2 w-100">13. Delete Old Enrollments</button>
```

```
    </div>
```

```
    <div class="col-md-6">
```

```
        <h4>Aggregation Tasks</h4>
```

```
        <button onclick="getEnglishAverage()" class="btn btn-secondary  
mb-2 w-100">14. English Average</button>
```

```
        <button onclick="getDepartmentCount()" class="btn btn-  
secondary mb-2 w-100">15. Department Count</button>
```

```
        <button onclick="getTopStudents()" class="btn btn-secondary  
mb-2 w-100">16. Top 3 Students</button>
```

```
    </div>
```

```
</div>
```

```
<div class="row mt-3">
  <div class="col-md-12">
    <h4>Indexing Tasks</h4>
    <button onclick="createNameIndex()" class="btn btn-dark mb-2">17. Create Name Index</button>
    <button onclick="showIndexes()" class="btn btn-dark mb-2">18. Show Indexes</button>
  </div>
</div>
</div>
</div>
```

```
<div class="card mb-4">
  <div class="card-header">
    <h4>Insert New Student</h4>
  </div>
  <div class="card-body">
    <form id="studentForm">
      <div class="row">
        <div class="col-md-4">
          <input type="number" class="form-control mb-2"
id="student_id" placeholder="Student ID (optional)">
          <input type="text" class="form-control mb-2" id="name"
placeholder="Name" required>
          <input type="number" class="form-control mb-2" id="age"
placeholder="Age" required>
        </div>
      </div>
    </form>
  </div>
</div>
```

```
<div class="col-md-4">
```

```
  <select class="form-control mb-2" id="gender" required>
```

```
    <option value="">Select Gender</option>
```

```
    <option value="Male">Male</option>
```

```
    <option value="Female">Female</option>
```

```
  </select>
```

```
  <select class="form-control mb-2" id="department" required>
```

```
    <option value="">Select Department</option>
```

```
    <option value="Computer Science">Computer  
Science</option>
```

```
    <option value="Data Science">Data Science</option>
```

```
    <option value="Electrical Engineering">Electrical  
Engineering</option>
```

```
    <option value="Mechanical Engineering">Mechanical  
Engineering</option>
```

```
    <option value="Civil Engineering">Civil  
Engineering</option>
```

```
    <option value="Chemical Engineering">Chemical  
Engineering</option>
```

```
  </select>
```

```
  <input type="date" class="form-control mb-2"  
id="enrollment_date" required>
```

```
</div>
```

```
<div class="col-md-4">
```

```
  <input type="number" class="form-control mb-2"  
id="mathMarks" placeholder="Math Marks (0-100)" required>
```

```
  <input type="number" class="form-control mb-2"  
id="scienceMarks" placeholder="Science Marks (0-100)" required>
```

```
        <input type="number" class="form-control mb-2"
id="englishMarks" placeholder="English Marks (0-100)" required>
    </div>
</div>
    <button type="submit" class="btn btn-success">Add New
Student</button>
</form>
</div>
</div>

<!-- Results Display -->
<div class="card">
    <div class="card-header">
        <h3>Results</h3>
    </div>
    <div class="card-body">
        <pre id="results" class="bg-light p-3 rounded"></pre>
    </div>
</div>
</div>

<script src="script.js"></script>
</body>
</html>
```

Style.css

```
.card {  
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);  
    margin-bottom: 20px;  
}
```

```
.card-header {  
    background-color: #f8f9fa;  
}
```

```
.btn {  
    margin-right: 5px;  
}
```

```
#results {  
    max-height: 400px;  
    overflow-y: auto;  
}
```

```
.form-control {  
    margin-bottom: 10px;  
}
```

```
/* Responsive adjustments */  
@media (max-width: 768px) {  
    .btn {
```

```
margin-bottom: 10px;  
width: 100%;  
}  
}
```

Script.js:

```
// Simulated database
```

```
let database = null;
```

```
let collection = null;
```

```
let indexes = new Set();
```

```
// Sample student data
```

```
const sampleStudents = [  
  {
```

```
    student_id: 1,
```

```
    name: "John Smith",
```

```
    age: 20,
```

```
    gender: "Male",
```

```
    department: "Computer Science",
```

```
    marks: {
```

```
      Math: 85,
```

```
      Science: 78,
```

```
      English: 92
```

```
    },
```

```
    enrollment_date: "2024-01-15"
```



```
},  
{  
  student_id: 2,  
  name: "Emma Wilson",  
  age: 22,  
  gender: "Female",  
  department: "Electrical Engineering",  
  marks: {  
    Math: 92,  
    Science: 88,  
    English: 85  
  },  
  enrollment_date: "2024-01-02"  
},  
{  
  student_id: 3,  
  name: "Michael Chen",  
  age: 21,  
  gender: "Male",  
  department: "Computer Science",  
  marks: {  
    Math: 78,  
    Science: 82,  
    English: 88  
  },  
  enrollment_date: "2024-01-10"
```

},

{

student_id: 4,

name: "Sarah Johnson",

age: 19,

gender: "Female",

department: "Data Science",

marks: {

Math: 95,

Science: 90,

English: 87

},

enrollment_date: "2024-01-20"

},

{

student_id: 5,

name: "David Brown",

age: 23,

gender: "Male",

department: "Mechanical Engineering",

marks: {

Math: 75,

Science: 68,

English: 82

},

enrollment_date: "2024-01-05"

```
},  
{  
  student_id: 6,  
  name: "Lisa Anderson",  
  age: 20,  
  gender: "Female",  
  department: "Computer Science",  
  marks: {  
    Math: 88,  
    Science: 85,  
    English: 90  
  },  
  enrollment_date: "2024-01-12"  
},  
{  
  student_id: 7,  
  name: "James Wilson",  
  age: 22,  
  gender: "Male",  
  department: "Electrical Engineering",  
  marks: {  
    Math: 82,  
    Science: 75,  
    English: 78  
  },  
  enrollment_date: "2024-01-01"
```

```
},  
{  
  student_id: 8,  
  name: "Emily Davis",  
  age: 21,  
  gender: "Female",  
  department: "Data Science",  
  marks: {  
    Math: 90,  
    Science: 92,  
    English: 85  
  },  
  enrollment_date: "2024-01-08"  
},  
{  
  student_id: 9,  
  name: "Robert Taylor",  
  age: 20,  
  gender: "Male",  
  department: "Computer Science",  
  marks: {  
    Math: 85,  
    Science: 80,  
    English: 88  
  },  
  enrollment_date: "2024-01-18"
```

```

    },
    {
        student_id: 10,
        name: "Maria Garcia",
        age: 22,
        gender: "Female",
        department: "Mechanical Engineering",
        marks: {
            Math: 78,
            Science: 85,
            English: 92
        },
        enrollment_date: "2024-01-04"
    }
];

// Helper function to display results
function displayResults(results) {
    document.getElementById('results').innerHTML = JSON.stringify(results,
    null, 2);
}

// Initialize database with sample data
function initializeDatabase() {
    database = { name: 'StudentRecords' };
    collection = [...sampleStudents];
    displayResults({

```

```
        message: 'Database initialized with sample data',
        studentCount: collection.length
    });
}

// Phase 1: Database & Collection Creation
function createDatabase() {
    database = { name: 'StudentRecords' };
    displayResults({ message: 'Database StudentRecords created successfully'
});
}

function createCollection() {
    if (!database) {
        displayResults({ error: 'Please create database first' });
        return;
    }
    collection = [];
    displayResults({ message: 'Collection Students created successfully' });
}

// Phase 2: Query Operations
function getAllStudents() {
    if (!collection) return displayResults({ error: 'Collection not found' });
    displayResults(collection);
}
```

```
function getCSStudents() {  
  if (!collection) return displayResults({ error: 'Collection not found' });  
  const csStudents = collection.filter(s => s.department.toLowerCase() ===  
'computer science');  
  displayResults(csStudents);  
}
```

```
function getHighMathScorers() {  
  if (!collection) return displayResults({ error: 'Collection not found' });  
  const highScorers = collection.filter(s => s.marks.Math > 80);  
  displayResults(highScorers);  
}
```

```
function getOlderStudents() {  
  if (!collection) return displayResults({ error: 'Collection not found' });  
  const olderStudents = collection.filter(s => s.age > 21);  
  displayResults(olderStudents);  
}
```

```
function getRecentEnrollments() {  
  if (!collection) return displayResults({ error: 'Collection not found' });  
  const recentStudents = collection.filter(s => s.enrollment_date > '2024-01-  
05');  
  displayResults(recentStudents);  
}
```

```
// Update Operations
```

```
function increaseMathMarks() {  
  if (!collection) return displayResults({ error: 'Collection not found' });  
  collection.forEach(s => s.marks.Math += 5);  
  displayResults({ message: 'Math marks increased by 5 for all students',  
collection });  
}
```

```
function addActiveStatus() {  
  if (!collection) return displayResults({ error: 'Collection not found' });  
  collection.forEach(s => s.status = 'active');  
  displayResults({ message: 'Active status added to all students', collection });  
}
```

```
function changeDepartment() {  
  if (!collection) return displayResults({ error: 'Collection not found' });
```

```
  // List of all possible departments
```

```
  const departments = [  
    'Computer Science',  
    'Data Science',  
    'Electrical Engineering',  
    'Mechanical Engineering',  
    'Civil Engineering',  
    'Chemical Engineering'
```

```
  ];
```

```
  // Randomly select a student
```



```
const randomIndex = Math.floor(Math.random() * collection.length);
const student = collection[randomIndex];

// Get current department
const currentDept = student.department;

// Filter out the current department and randomly select a new one
const availableDepts = departments.filter(dept => dept !== currentDept);
const newDept = availableDepts[Math.floor(Math.random() *
availableDepts.length)];

// Update the student's department
student.department = newDept;

displayResults({
  message: `Changed department for student ${student.name}`,
  from: currentDept,
  to: newDept,
  updatedStudent: student
});
}

// Delete Operations
function deleteLowestScience() {
  if (!collection) return displayResults({ error: 'Collection not found' });
  if (collection.length === 0) return displayResults({ error: 'Collection is
empty' });
}
```

```
const lowestScience = collection.reduce((min, student) =>
  student.marks.Science < min.marks.Science ? student : min
);

collection = collection.filter(s => s.student_id !==
lowestScience.student_id);

displayResults({ message: 'Student with lowest science marks deleted',
deletedStudent: lowestScience });
}
```

```
function deleteOldEnrollments() {
  if (!collection) return displayResults({ error: 'Collection not found' });
  const originalLength = collection.length;
  collection = collection.filter(s => s.enrollment_date >= '2024-01-03');
  displayResults({
    message: `${originalLength - collection.length} students deleted`,
    remainingStudents: collection
  });
}
```

// Aggregation Operations

```
function getEnglishAverage() {
  if (!collection) return displayResults({ error: 'Collection not found' });
  if (collection.length === 0) return displayResults({ error: 'Collection is
empty' });
}
```

```
    const avg = collection.reduce((sum, student) => sum +
student.marks.English, 0) / collection.length;

    displayResults({ averageEnglishMarks: avg.toFixed(2) });
}
```

```
function getDepartmentCount() {
    if (!collection) return displayResults({ error: 'Collection not found' });
    const deptCount = collection.reduce((acc, student) => {
        acc[student.department] = (acc[student.department] || 0) + 1;
        return acc;
    }, {});
    displayResults(deptCount);
}
```

```
function getTopStudents() {
    if (!collection) return displayResults({ error: 'Collection not found' });
    const studentsWithTotal = collection.map(student => ({
        ...student,
        totalMarks: student.marks.Math + student.marks.Science +
student.marks.English
    }))
    .sort((a, b) => b.totalMarks - a.totalMarks)
    .slice(0, 3);
    displayResults(studentsWithTotal);
}
```

// Indexing Operations

```
function createNameIndex() {  
  if (!collection) return displayResults({ error: 'Collection not found' });  
  indexes.add('name');  
  displayResults({ message: 'Index created on name field' });  
}
```

```
function showIndexes() {  
  if (!collection) return displayResults({ error: 'Collection not found' });  
  displayResults({ indexes: Array.from(indexes) });  
}
```

```
// Handle form submission for inserting new student records  
document.addEventListener('DOMContentLoaded', function() {  
  document.getElementById('studentForm').addEventListener('submit',  
function(e) {  
  e.preventDefault();  
  
  if (!collection) {  
    displayResults({ error: 'Please initialize database first' });  
    return;  
  }  
  
  // Get the next available student ID  
  const maxId = Math.max(...collection.map(s => s.student_id));  
  const newId = document.getElementById('student_id').value || (maxId +  
1);
```

```
// Check if ID already exists
if (collection.some(s => s.student_id === parseInt(newId))) {
    displayResults({ error: 'Student ID already exists' });
    return;
}

const newStudent = {
    student_id: parseInt(newId),
    name: document.getElementById('name').value,
    age: parseInt(document.getElementById('age').value),
    gender: document.getElementById('gender').value,
    department: document.getElementById('department').value,
    marks: {
        Math: parseInt(document.getElementById('mathMarks').value),
        Science: parseInt(document.getElementById('scienceMarks').value),
        English: parseInt(document.getElementById('englishMarks').value)
    },
    enrollment_date: document.getElementById('enrollment_date').value
};

// Add the new student to the collection
collection.push(newStudent);

// Display success message
displayResults({
    message: 'New student added successfully',
```

Output:

Phase 1: Database & Data Creation

Initialize Database with Sample Data

Phase 2: Queries

Retrieval Tasks

4. Get All Students

5. Get CS Students

6. Math Score > 80

7. Age > 21

8. Recent Enrollments

Delete Tasks

12. Delete Lowest Science

13. Delete Old Enrollments

Update Tasks

9. Increase Math Marks

10. Add Active Status

11. Change Department

Aggregation Tasks

14. English Average

15. Department Count

16. Top 3 Students

Delete Tasks

12. Delete Lowest Science

13. Delete Old Enrollments

Aggregation Tasks

14. English Average

15. Department Count

16. Top 3 Students

Indexing Tasks

17. Create Name Index

18. Show Indexes

Insert New Student

Student ID (optional)

Select Gender

Math Marks (0-100)

Name

Select Department

Science Marks (0-100)

Age

dd-mm-yyyy

English Marks (0-100)

Add New Student

Results

